

به نام خالق زیبایی‌ها

مینی پروژه بوت کمپ پایتون آلفا
سری سیزدهم بوت کمپ‌های کوئرا
تابستان ۱۴۰۵

BioForge

خط لوله جامع تحلیل داده های ژنتیکی



معرفی پروژه (Project Introduction)

BioForge یک پروژه است که هدف آن ساخت یک Pipeline خط فرمانی (CLI) برای پردازش DNA Sequence ها با Python است. هدف تمرین طراحی درست، تمیز و قابل دفاع در مقیاسی کوچک است. از Over Engineering خودداری کنید.

Pipeline باید مراحل زیر را به ترتیب انجام دهد:

FASTA Input



Parsing



Validation



ORF Detection



Translation



Filtering



Annotation



Reporting



Logging

هر مرحله در بخش‌های بعدی این سند به‌طور کامل و دقیق مشخص شده است؛ Implementation نهایی بر عهده شماست.

❖ ورودی پروژه

پروژه یک فایل **FASTA** چند رکوردی می‌گیرد. مثال:

```
>seq001 organism=E_coli sample=A  
ATGCTTTCATAG
```

```
>seq002 organism=Human sample=B  
CCCATGGGGTAA
```

هر **Record** شامل سه بخش است:

- ID
- Description
- Sequence

ID فقط اولین بخش Header است. مثلاً برای `<seq001 organism=E_coli sample=A`، مقدار ID می‌شود `seq001` و بقیه Header، Description است.

اگر در Header مقدار `...=organism` وجود داشت، این مقدار هم باید استخراج شود.

❖ FASTA Parsing

باید بخشی برای خواندن فایل FASTA بنویسید. این بخش باید:

- چند Record را بخواند.
- خطوط خالی را نادیده بگیرد.
- حروف کوچک را هم بپذیرد.
- فایل خالی را تشخیص دهد.
- اگر قبل از اولین < توالی (Sequence) وجود داشت، آن را خطا بداند.
- Header بدون Sequence را تشخیص دهد.
- اگر ID تکراری دیده شد، آن را به صورت Warning در Log ثبت کند.

DNA Sequence ❖

1. DNA چیست؟

DNA یا **Deoxyribonucleic Acid** مولکولی است که اطلاعات ژنتیکی را ذخیره می‌کند.

در این پروژ، DNA را به صورت یک Sequence از چهار **Nucleotide Base** در نظر می‌گیریم:

A → Adenine

T → Thymine

C → Cytosine

G → Guanine

بنابراین یک DNA Sequence معتبر فقط می‌تواند شامل **A، C، G** و **T** باشد.

مثال معتبر:

ATGCTACG

مثال نامعتبر:

ATGXYZ

در صورت نامعتبر بودن Sequence، باید Exception اختصاصی **InvalidSequenceError** ایجاد شود.

Sequence نامعتبر نباید با حذف یا جایگزین کردن کاراکترها به صورت خودکار اصلاح شود.

Complement .2

هر Base در DNA یک Base مکمل دارد:

$A \leftrightarrow T$

$C \leftrightarrow G$

برای مثال:

DNA: ATGC

Complement: TACG

در Complement ترتیب Sequence تغییر نمی‌کند و فقط هر Base با مکمل خودش جایگزین می‌شود.

Reverse Complement .3

Reverse Complement شامل دو عملیات است:

1. پیدا کردن Complement

2. Reverse کردن Sequence حاصل

مثال:

DNA: ATGC

Complement: TACG

Reverse Complement: GCAT

DNA → RNA .4

در RNA به جای **Thymine T** از **Uracil U** استفاده می‌شود.

در این پروژه تبدیل به صورت مستقیم انجام می‌شود:

A → A

T → U

C → C

G → G

مثال:

DNA: ATGC

RNA: AUGC

در این پروژه نیازی به شبیه‌سازی کامل فرآیند زیستی Transcription نیست.

GC Content .5

GC Content درصد Base های **G** و **C** در یک DNA Sequence است.

فرمول:

$$\text{GC Content} = (\text{Number of G} + \text{Number of C}) / \text{Total Length} \times 100$$

مثال:

DNA: ATGC

G = 1

C = 1

Length = 4

6. نکته طراحی

عملیات‌های زیر باید در ساختار برنامه پشتیبانی شوند:

- Validation
- Complement
- Reverse Complement
- DNA → RNA
- GC Content

لازم نیست برای هر عملیات یک Class جداگانه ایجاد کنید.

هدف این بخش، استفاده معنادار از OOP و مشخص بودن مسئولیت هر بخش است؛ بنابراین قبل از ایجاد هر Class باید بتوانید دلیل وجود و مسئولیت آن را توضیح دهید.

❖ فایل‌های Data

برای اجرای بخش‌های مربوط به **Translation** و محاسبه وزن مولکولی پروتئین، دو فایل داده در اختیار شما قرار داده شده است:

- **codon_table.txt** – شامل جدول کامل کدون‌ها و آمینواسید متناظر با هر کدون
- **amino_weights.txt** – شامل کد یک‌حرفی آمینواسیدها و وزن مولکولی آن‌ها

این فایل‌ها را در مسیر زیر داخل پروژه قرار دهید:

```
BioForge/  
└── data/  
    ├── codon_table.txt  
    └── amino_weights.txt
```

نکته: داده‌های موجود در این فایل‌ها را مستقیماً داخل کد Hardcode نکنید. برنامه باید این اطلاعات را از فایل‌های داده بخواند و از آن‌ها استفاده کند.

مثال محتوای Codon Table:

AUG M

UUU F

UAA *

مثال محتوای Amino Weights:

A 71.037

C 103.009

M 131.040

اگر خطی از این فایل‌ها خراب بود:

- باید خطا ایجاد شود.
- شماره خط خراب مشخص شود.

Translation ❖

Translation (ترجمه) فرایندی است که در آن اطلاعات موجود در RNA پیام‌رسان (mRNA) به یک زنجیره پروتئینی (Protein) تبدیل می‌شود.

در سلول، این فرایند توسط Ribosome انجام می‌شود. ریبوزوم mRNA را از روی Codon‌های سه‌نوکلئوتیدی می‌خواند و بر اساس اطلاعات هر Codon، Amino Acid مناسب را به زنجیره‌ی در حال تشکیل اضافه می‌کند.

Codon چیست؟

هر Codon شامل سه نوکلئوتید از RNA است. هر Codon معمولاً مشخص می‌کند که کدام Amino Acid باید در زنجیره‌ی پروتئینی قرار بگیرد.

برای مثال:

AUG → Methionine (M)

CUU → Leucine (L)

UCA → Serine (S)

بنابراین:

AUG CUU UCA

به:

M L S

ترجمه می‌شود.

Start Codon

AUG علاوه بر اینکه Codon مربوط به Methionine است، معمولاً به‌عنوان Start Codon نیز شناخته می‌شود و شروع فرایند ترجمه را مشخص می‌کند.

در این پروژه، ORF هایی که پیدا می‌کنید از AUG شروع می‌شوند؛ بنابراین ترجمه‌ی ORF از همان Codon شروع می‌شود.

Stop Codon

سه Codon زیر:

UAA ، UAG ، UGA

کدکنده‌ی Amino Acid نیستند. این Codon ها به‌عنوان Stop Codon عمل می‌کنند و پایان ترجمه را مشخص می‌کنند.

بنابراین Stop Codon نباید به‌عنوان یک حرف یا Amino Acid در Protein نهایی قرار بگیرد.

مثلاً:

AUG CUU UCA UAG

به شکل زیر ترجمه می‌شود:

AUG → M

CUU → L

UCA → S

UAG → Stop

بنابراین Protein نهایی:

MLS

است، نه چیزی که شامل Stop Codon باشد.

ارتباط Translation با ORF

در بخش ORF، شما ناحیه‌ای از RNA را پیدا می‌کنید که از AUG شروع شده و در صورت کامل بودن، به یک Stop Codon می‌رسد.

در مرحله‌ی Translation، همان Codon ها به Amino Acid تبدیل می‌شوند تا Protein مربوط به ORF به دست بیاید.

به صورت مفهومی:

RNA → Codons → Amino Acids → Protein

پس خروجی Translation، یک رشته از Amino Acid هاست که نمایانگر پروتئین حاصل از ORF است.

ORF



در این بخش باید ORF Open Reading Frame های موجود در توالی را پیدا کنید.

ORF چیست؟

ORF بخشی از توالی RNA است که با Start Codon یعنی AUG شروع می‌شود و در حالت کامل، با یکی از Stop Codon های زیر به پایان می‌رسد:

UAA ، UAG ، UGA

از آنجا که هر Codon از ۳ نوکلئوتید تشکیل شده است، بعد از پیدا کردن AUG باید توالی را سه‌تا سه‌تا و در همان Reading Frame بررسی کنید تا به Stop Codon برسید.

برای مثال:

AUG | GCU | AAA | UGA

در این مثال AUG شروع و UGA پایان ORF است؛ بنابراین ORF کامل است.

Reading Frame

یک توالی را می‌توان از سه موقعیت مختلف خواند. این سه حالت عبارت‌اند از:

- Frame 0
- Frame 1
- Frame 2

بنابراین باید هر سه Frame در جهت Forward و هر سه Frame در جهت Reverse بررسی شوند.

در مجموع ۶ Reading Frame داریم:

- Forward Frame 0
- Forward Frame 1
- Forward Frame 2
- Reverse Frame 0
- Reverse Frame 1
- Reverse Frame 2

اهمیت Frame Alignment

صرفاً پیدا کردن AUG و اولین Stop Codon بعد از آن کافی نیست.

Stop Codon باید دقیقاً در همان Reading Frame قرار داشته باشد. بنابراین بعد از پیدا شدن AUG، ادامه‌ی توالی باید با حفظ همان Alignment و به صورت Codon های سه‌تایی بررسی شود.

استفاده از Regex می‌تواند برای پیدا کردن الگوها کمک‌کننده باشد، اما Regex به‌تنهایی برای تشخیص صحیح ORF کافی نیست و باید منطق Reading Frame و Codon Alignment نیز در نظر گرفته شود.

ORF ناقص

ممکن است یک AUG پیدا شود، اما در ادامه و در همان Reading Frame هیچ Stop Codon معتبری وجود نداشته باشد.

در این حالت ORF را حذف نکنید و آن را به عنوان ORF ناقص ثبت کنید:

```
is_complete = False
```

اگر Stop Codon مناسب پیدا شود:

```
is_complete = True
```

بنابراین هم ORF های کامل و هم ORF های ناقص باید در خروجی وجود داشته باشند.

Reverse Strand

برای بررسی Reverse Strand، فقط Reverse کردن Sequence کافی نیست.

ابتدا باید Reverse Complement توالی را بسازید.

در DNA، نوکلئوتید ها به صورت زیر مکمل یکدیگر هستند:

A ↔ T

C ↔ G

برای ساخت Reverse Complement، ابتدا Complement توالی را به دست آورید و سپس نتیجه را Reverse کنید.

بعد از ساخت Reverse Complement، سه Reading Frame مربوط به Reverse Strand را نیز بررسی کنید.

Reverse Strand در start_pos

اگر یک ORF در Reverse Strand پیدا شود، موقعیتی که هنگام بررسی Reverse به دست می‌آورید مربوط به Reverse Strand است.

اما مقدار نهایی start_pos باید بر اساس DNA اصلی (Original Strand) باشد.

بنابراین باید Coordinate مربوط به Reverse را به Coordinate متناظر آن در DNA اصلی تبدیل کنید.

در نتیجه، start_pos نهایی برای تمام ORF ها باید نسبت به Original DNA گزارش شود.

اطلاعات مورد نیاز برای هر ORF

برای هر ORF باید اطلاعات زیر را نگه دارید:

- Strand: جهت Forward یا Reverse
- Frame: یکی از Frame های 0، 1 یا 2
- start_pos: موقعیت شروع ORF بر اساس DNA اصلی
- Protein: پروتئین حاصل از ترجمه ORF
- is_complete: مشخص می‌کند ORF کامل است یا ناقص

هدف این بخش این است که ORF ها را با در نظر گرفتن Reading Frame، Codon Alignment، Reverse Complement و مختصات DNA اصلی به درستی شناسایی و ثبت کنید.

ORF Class ❖

برای هر ORF این اطلاعات باید نگهداری شود:

- Protein
- Strand
- Frame
- Start Position
- Complete / Incomplete
- بهتر است Protein به صورت یک Protein Sequence نگهداری شود.

Filter ها ❖

پس از انجام Pipeline، باید امکان فیلتر کردن Protein ها بر اساس معیارهای مختلف وجود داشته باشد.

سه Filter موردنیاز است:

1. LengthFilter

این Filter بر اساس طول Protein عمل می‌کند. طول Protein برابر با تعداد اسیدآمینوهای موجود در Sequence آن است.

برای مثال:

--4-length-min

یعنی فقط Protein هایی انتخاب شوند که طول آن‌ها حداقل ۴ اسید آمینه باشد.

2. WeightFilter

این Filter بر اساس وزن مولکولی (Molecular Weight) پروتئین عمل می‌کند.

وزن مولکولی Protein باید بر اساس اسیدآمینه‌های تشکیل‌دهنده آن محاسبه شود و سپس Protein ها بر اساس شرط یا محدوده وزنی مشخص شده فیلتر شوند.

3. MotifFilter

این Filter بر اساس Motif های پیدا شده در Protein عمل می‌کند.

Motif یک الگوی مشخص از اسیدآمینه‌ها در یک Sequence است. در این پروژه، فقط پیدا کردن Motif کافی نیست؛ نام یا Sequence مربوط به Motif و موقعیت آن در Protein نیز باید در نتایج قابل دسترسی باشد.

برای مثال:

Motif: TAG

Position: 4

این اطلاعات باید به شکلی در دسترس باشد که MotifFilter بتواند از آن برای فیلتر کردن Protein ها استفاده کند.

نکته مهم:

پیدا کردن Motif و فیلتر کردن بر اساس Motif دو مرحله جدا هستند. ابتدا Motif باید در مرحله تحلیل Sequence شناسایی شود و اطلاعات آن (شامل Motif و موقعیت آن) نگهداری شود. سپس MotifFilter از این اطلاعات برای فیلتر کردن نتایج استفاده می‌کند.

OOP ❖

این بخش خیلی مهم است.

پروژه باید از OOP استفاده کند، اما لازم نیست همه چیز Class باشد.

این مفاهیم باید در پروژه دیده شوند:

- Encapsulation
- Inheritance
- Polymorphism
- Composition

شما باید خودتان تصمیم بگیرید کدام بخش Class باشد و کدام بخش Function.

در README این موارد را توضیح دهید:

- چرا این بخش را Class کردید؟
- چرا این بخش را Function کردید؟
- Inheritance کجا استفاده شده؟
- Polymorphism کجا استفاده شده؟
- Composition کجا استفاده شده؟

نکته مهم: تعداد Class بیشتر به معنی طراحی بهتر نیست.

❖ Exception ها

این Exception ها باید وجود داشته باشند:

BioForgeError

└─ FastaFormatError

└─ InvalidSequenceError

└─ DataFileError

کاربرد هر کدام:

- FastaFormatError: برای خطاهای ساختار FASTA.
- InvalidSequenceError: برای DNA نامعتبر.
- DataFileError: برای خطاهای Data File.

اگر یک Record خراب بود، در صورت امکان نباید کل Pipeline متوقف شود. Record خراب باید Log شود و بقیه Record های سالم ادامه پیدا کنند.

❖ Logging

باید یک فایل Log داشته باشید:

output/bioforge.log

این موارد باید در آن ثبت شوند:

- Record خراب
- Sequence نامعتبر
- ID تکراری
- Warning ها

- خطاهای Data File

Log باید به صورت Append نوشته شود (نه Overwrite).

Annotation ❖

برای ORF های نهایی باید ID خودکار بسازید:

BFG_001

BFG_002

BFG_003

...

خروجی ❖

این فایل ها باید تولید شوند:

output/report.txt

output/bioforge.log

report.txt حداقل باید شامل این موارد باشد:

- ID
- Strand
- Frame
- Start Position
- Protein
- Complete / Incomplete

CLI ❖

برنامه باید با argparse اجرا شود. مثال اجرا:

```
python main.py --input input/input.fasta --out output/ --min-length 4
```

این گزینه‌ها اجباری هستند:

- input--
- out--
- min-length--

--min-length باید واقعاً روی Filtering اثر بگذارد.

File Handling ❖

برای باز کردن فایل‌ها از این الگو استفاده کنید:

```
with open(..., encoding="utf-8")
```

این موارد باید مدیریت شوند:

- `FileNotFoundError`
- فایل خالی
- فایل خراب
- Data File نامعتبر
- FASTA نامعتبر

نکته مهم درباره طراحی

این پروژه یک تمرین OOP است، اما هدف تبدیل کردن همه چیز به Class نیست.

باید بتوانید برای انتخاب‌های طراحی خود دلیل بیاورید.

مثال (فقط برای روشن شدن موضوع، نه راه حل قطعی):

- شاید Header Parsing یک Function باشد، چون فقط یک ورودی می‌گیرد و State خاصی ندارد.
- شاید DNA Sequence یک Class باشد، چون داده و رفتار مرتبط با Sequence را با هم نگه می‌دارد.

طراحی نهایی را خودتان انتخاب کنید. در README حداقل ۵ Design Decision را توضیح دهید.

خروجی نهایی پروژه

با اجرای:

```
python main.py --input input/input.fasta --out output/ --min-length 4
```

برنامه باید بتواند:

1. فایل FASTA را بخواند.
2. Sequence ها را بررسی (Validate) کند.
3. ORF های هر ۶ Frame را پیدا کند.
4. Forward و Reverse Strand را بررسی کند.
5. ORF های Complete و Incomplete را تشخیص دهد.
6. ORF ها را Translate کند.
7. Filter ها را اجرا کند.
8. ORF ها را Annotate کند.
9. Report ها را تولید کند.
10. خطاها و Warning ها را در Log ثبت کند.

موفق و بی‌باگ باشید!!!